Divide and Conquer Notes

1 Recurrence Relations

Recurrence relations provide an upper bound for the running time of divide and conquer algorithms based on the running time on smaller instances. Given a problem of input size n with running time $T(n)$, the general form of such recurrences is:

$$T(n) \leq qT(\frac{n}{b}) + cn^\alpha \qquad \text{for } n > b$$
$$T(n) \leq c \qquad \text{for } n \leq b$$

where q is the number of subproblems, $\frac{n}{b}$ is the size of each subproblem, and cn^α (for some constant c) is the amount of work done outside of the recursive calls.

1.1 Master Theorem

Given a recurrence $T(n) \leq qT(\frac{n}{b}) + cn^\alpha$:

- $q > b^\alpha \Rightarrow T(n) = O(n^{\log_b q})$
- $q = b^\alpha \Rightarrow T(n) = O(n^\alpha \log_b n)$
- $q < b^\alpha \Rightarrow T(n) = O(n^\alpha)$

1.2 Bounding Recurrences

How were the running times from (1.1) derived? The most intuitive way to approach this is to imagine a "recursion tree": first, find the amount of work being done on each level of this tree, then find the total number of levels. In general, the tree will have $O(\log_b n)$ levels, and

$$\begin{aligned}\text{cost for level } \ell &= (\#\text{ of subproblems}) \cdot (\text{cost per subproblem}) \\ &= q^\ell \cdot c(\frac{n}{b^\ell})^\alpha = cn^\alpha(\frac{q}{b^\alpha})^\ell,\end{aligned}$$

so the overall running time (of course, this usually simplifies nicely) will be of the form

$$T(n) = \sum_{\ell=0}^{\log_b n} cn^\alpha(\frac{q}{b^\alpha})^\ell = cn^\alpha \sum_{\ell=0}^{\log_b n} (\frac{q}{b^\alpha})^\ell.$$

Finally, to get the running time in terms of Big-Oh, note that $T(n)$ is actually a geometric series with

$$r = \frac{q}{b^\alpha}.$$

Now, we can reason about the behavior of each case using r, and derive the three cases of the master theorem exactly:

- $r > 1$: The largest term of the summation will be the last one, i.e.

  $$r^{\log_b n} = r^{\frac{\ln n}{\ln b}} = e^{\ln r(\frac{\ln n}{\ln b})} = n^{\log_b r}.$$

  Therefore,

  $$T(n) \approx cn^\alpha \cdot n^{\log_b r} = cn^{\alpha + \log_b r}.$$

  Plugging back $r = \frac{q}{b^\alpha}$:

  $$T(n) = cn^{\alpha + \log_b q - \alpha} = O(n^{\log_b q}).$$

- $r = 1$: Here, we're just summing $r = 1$ $\log_b n$ times, so the running time is $O(n^\alpha \log n)$.
- $r < 1$: The largest term of the summation is the first one, i.e. $r^0 = 1$, so the running time is $O(n^\alpha)$.

2 Known Problems

2.1 Merge Sort

Divide the input into two sub-lists of equal size, solve the two subproblems recursively, and then merge the sub-lists in linear time. Clearly, the recurrence is $T(n) \leq 2T(\frac{n}{2}) + cn$ with running time $O(n \log n)$.

Recurrence Bound. Assuming that the root of the tree is level 0, the number of subproblems on level ℓ is 2^ℓ, with the size of each subproblem being $\frac{n}{2^\ell}$. Since merging two halves cost linear time, the total amount of work being done on each level is $c \cdot 2^\ell(\frac{n}{2^\ell}) = cn$. Moreover, as we split the problem in half each time, the tree has $O(\log n)$ levels. Thus, the overall running time is $O(n \log n)$.

2.2 Integer Multiplication

The naive way of integer multiplication (i.e. the method we learn in school) costs $O(n^2)$. However, note that we can split up an n-digit integer x like so:

$$x = x_1 \cdot 10^{\frac{n}{2}} + x_0.$$

Using this, we can rewrite the multiplication of two n-digit integers xy as:

$$\begin{aligned} xy &= (x_1 \cdot 10^{\frac{n}{2}} + x_0)(y_1 \cdot 10^{\frac{n}{2}} + y_0) \\ &= x_1 y_1 \cdot 10^n + (x_1 y_0 + x_0 y_1) \cdot 10^{\frac{n}{2}} + x_0 y_0. \end{aligned}$$

Now, we only need to compute four products of two $\frac{n}{2}$-digit integers. Since integer addition costs linear time, the recurrence is $T(n) \leq 4T(\frac{n}{2}) + cn$ with running time $O(n \log n)$.

Extension: Karatsuba's Algorithm. There is a trick we can use to get the number of subproblems down to three. We can find $x_1 y_0 + x_0 y_1$ like so:

$$\begin{aligned} (x_0 + x_1)(y_0 + y_1) &= x_0 y_0 + x_0 y_1 + x_1 y_0 + x_1 y_1 \\ \Rightarrow x_1 y_0 + x_0 y_1 &= (x_0 + x_1)(y_0 + y_1) - x_0 y_0 - x_1 y_1. \end{aligned}$$

Then, the only products we need to compute are $x_0 y_0, x_1 y_1$, and $(x_0 + x_1)(y_0 + y_1)$, so our recurrence becomes $T(n) \leq 3T(\frac{n}{2}) + cn$, with running time $O(n^{\log_2 3}) \approx O(n^{1.58})$.

2.3 Matrix Multiplication

The naive way of multiplying two $n \times n$ matrices costs $O(n^3)$. Let us consider an initial divide and conquer strategy: given the product $AB = C$, we can split each matrix as follows:

$$\begin{bmatrix} \begin{bmatrix} A_{11} \\ A_{21} \end{bmatrix} & \begin{bmatrix} A_{12} \\ A_{22} \end{bmatrix} \end{bmatrix} \cdot \begin{bmatrix} \begin{bmatrix} B_{11} \\ B_{21} \end{bmatrix} & \begin{bmatrix} B_{12} \\ B_{22} \end{bmatrix} \end{bmatrix} = \begin{bmatrix} \begin{bmatrix} A_{11}B_{11} + A_{12}B_{21} \\ A_{21}B_{11} + A_{22}B_{21} \end{bmatrix} & \begin{bmatrix} A_{11}B_{12} + A_{12}B_{22} \\ A_{21}B_{12} + A_{12}B_{22} \end{bmatrix} \end{bmatrix}$$

Note that adding two matrices together costs $O(n^2)$. Since we need to compute eight multiplications on two $\frac{n}{2} \times \frac{n}{2}$ matrices, the corresponding recurrence is $T(n) \leq 8T(\frac{n}{2}) + cn^2$, giving us a runtime of $O(n^3)$.

Strassen Algorithm. We can only reduce this runtime if we perform less than eight multiplications. Strassen discovered that we only have to perform seven (the details are omitted), giving us the improved bound of $T(n) \leq 7T(\frac{n}{2}) + cn^2$, with a running time of $O(n^{\log_2 7}) \approx O(n^{2.81})$.

2.4 Randomized Median Finding

Given a set of n numbers $S = \{a_1, ..., a_n\}$, how can we find the median (i.e. the kth largest element when $k = \frac{n}{2}$) without sorting the list? This randomized algorithm works as follows:

1. Uniformly sample an element $a_i \in S$ (the "pivot").
2. Partition S into:
$$S^- = \{\text{elements} < a_i\} \text{ and } S = \{\text{elements} > a_i\}.$$

3. If $|S^-| + 1 = k$, then the kth largest element must be a_i; return a_i.
   If $|S^-| \geq k$, then the kth largest element must be in S^-; recurse on $|S^-|$ with the same k.
   If $|S^-| < k$, then the kth largest element must be in S^+.; recurse on $|S^+|$ with an adjusted k.

To find the recurrence relation, we must find (1) the size of each subproblem, and (2) the work done outside of recursive calls. To find (1), we define a "good" pivot as one that is in the middle half of the list; that is, at least $\frac{n}{4}$ elements are larger than the pivot, and at least $\frac{n}{4}$ elements are smaller. By this definition, we can bound S^- and $S+$ as follows:

$$\frac{n}{4} \leq S^-, S^+ \leq \frac{3n}{4}.$$

So, on each recursive call, our new input size is at most $\frac{3n}{4}$. Next, to find (2), let us consider the expected number of times it takes before we sample a "good" pivot: the probability that the element is "good" is exactly one half, so it follows that

$$\mathbb{E}[\#\text{ tries before a "good" pivot}] = \frac{1}{\frac{1}{2}} = 2.$$

Therefore, our recurrence is $T(n) \leq T(\frac{3n}{4}) + 2n$, giving us a running time of $O(n)$.

3 Convolutions

A **convolution** is an operator that combines two vectors a and b to produce a third vector:

$$\begin{aligned} a * b &= \sum_{(i,j):i+j=k} a_i b_j \\ &= (a_0b_0, a_1b_0 + a_0b_1, \ldots, a_{n-1}b_{n-1}). \end{aligned}$$

The coordinate of the convoluted vector can be found by summing along each diagonal of this table:

$$\begin{array}{ccccc} a_0b_0 & a_0b_1 & \ldots & a_0b_{n-2} & a_0b_{n-1} \\ a_1b_0 & a_1b_1 & \ldots & a_1b_{n-2} & a_1b_{n-1} \\ a_2b_0 & a_2b_1 & \ldots & a_2b_{n-2} & a_2b_{n-1} \\ \ldots & \ldots & \ldots & \ldots & \ldots \\ a_{n-1}b_0 & a_{n-1}b_1 & \ldots & a_{n-2}b_{n-2} & a_{n-1}b_{n-1} \end{array}$$

3.1 Fast Fourier Transform (FFT)

Using the naive method, convoluting two n-dimensional vectors costs $O(n^2)$. However, we can improve this runtime using a divide and conquer strategy, namely the **Fast Fourier Transform.**

Given two vectors $a = (a_0, ..., a_{n-1})$ and $b = (b_0, ..., b_{n-1})$, we rewrite them as polynomials:

$$A(x) = a_0 + a_1x + ... + a_{n-1}x^{n-1}$$
$$B(x) = b_0 + b_1x + ... + b_{n-1}x^{n-1}$$

Then, computing $a * b$ is equivalent to computing the coefficients of $C(x) = A(x) \cdot B(x)$:

1. **Evaluate $A(x)$ and $B(x)$ at $2n$ values.** Notice that because $A(x)$ and $B(x)$ both have degree $n-1$, $C(x)$ will have degree at most $2n-2$, so we need $2n-1$ points to uniquely determine $C(x)$ (for convenience, we choose $2n$ points instead). To evaluate $A(x)$ and $B(x)$ efficiently at these points,, we can use the following identity to employ a divide and conquer strategy:

   $$A(x) = A_{\text{even}}(x^2) + x \cdot A_{\text{odd}}(x^2),$$

   where $A_{\text{even}}(x)$ contains the even-indexed coefficients and $A_{\text{odd}}(x)$ contains the odd-indexed coefficients of $A(x)$. Each of these polynomials has degree at most $\frac{n}{2} - 1$.

   How can we find evaluation points $x_0, ..., x_{2n-1}$ such that they relate some way to $x_0^2, ..., x_{2n-1}^2$? We consider a special group of complex numbers: first, a **complex k-th root of unity** is defined as any complex number z such that

   $$z^k = 1.$$

Then, the $2n$**-th roots of unity** are defined as:

$$\omega = e^{\frac{2\pi}{2n}i}, w^j = e^{\frac{2\pi j}{2n}i}.$$

(Recall since $e^{2\pi i} = 1$, it holds that $\omega^j = (e^{\frac{2\pi j}{2n}i})^{2n} = (e^{2\pi i})^j = 1$.)

Therefore, if we choose $1, \omega, \omega^2, ..., \omega^{2n-1}$ as our evaluation points, we have:

$$A(\omega^j) = A_{\text{even}}(\omega^{2j}) + x \cdot A_{\text{odd}}(\omega^{2j}),$$

where ω^{2j} are the n-th roots of unity and only takes on n distinct values. Thus, evaluating $A(x)$ at $2n$ points reduces to evaluating $A_{\text{even}}(x)$ and $A_{\text{odd}}(x)$ at n points. This gives us the recurrence $T(n) = 2T(\frac{n}{2}) + cn$, which is a running time of $O(n \log n)$.

2. **Compute $C(x)$.** For each of the $2n$ points, calculate $C(x) = A(x) \cdot B(x)$, which costs $O(n)$.
3. **Recover $C(x)$ from $C(1), C(\omega), ..., C(w^{2n-1})$.** We want to recover the coefficients of $C(x)$ from these values. Essentially, we perform inverse FFT, so this again costs $O(n \log n)$.

More applications. Apart from polynomial multiplication (discussed above), convolutions are also used to calculate the joint probability of two independent random variables X and Y:

$$Pr(X + Y = k) = \sum_{(i,j):i+j=k} Pr(X = i) \cdot Pr(Y = j).$$